Software Requirements Specification

Deep Retina

Diabetic Retinopathy Severity Grading System

Assignment 7 — Capstone Mini-Project (ML / DL)
Artificial Intelligence (Machine Learning & Deep Learning)

Name: Muhammad Shahmir Raza

Date: 27-August-2026

Document Version: 1.0

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements, tools, and system design of "Deep Retina" — a deep learning system that grades diabetic retinopathy (DR) severity from a retinal fundus photograph. It is written to satisfy the SRS deliverable for Assignment 7 of the NAVTTC Artificial Intelligence (Machine Learning & Deep Learning) capstone project, and to document the system for future reference, extension, or evaluation.

1.2 Project Scope

The system accepts a single retinal fundus image uploaded by a user through a web interface, preprocesses it, and passes it through a trained Convolutional Neural Network (CNN) to classify the diabetic retinopathy severity into one of five ordinal grades (No DR, Mild, Moderate, Severe, or Proliferative DR). It then presents the prediction, a model confidence score, and several supporting visualizations to the user through a Streamlit web application. The project covers the complete machine-learning lifecycle: dataset exploration, preprocessing, model training and evaluation, and deployment as an interactive demo.

This system is built strictly as an educational capstone project. It is not a certified medical device and is not intended to be used for actual clinical diagnosis or treatment decisions.

1.3 Intended Audience

- Course instructors and evaluators assessing the capstone project.
- Developers who may extend, retrain, or maintain the system in the future.
- Recruiters or portfolio reviewers evaluating the project as a demonstration of applied ML/DL skills.
- End users trying the public demo to understand what the model can do.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
DR	Diabetic Retinopathy — a diabetes complication that damages the retina's blood vessels.
CNN	Convolutional Neural Network — a deep learning architecture suited to image data.
EDA	Exploratory Data Analysis — analyzing dataset characteristics before modeling.
QWK	Quadratic Weighted Kappa — the official APTOS competition metric; measures agreement between predicted and true ordinal grades, penalizing larger errors more heavily.
APTOS	Asia Pacific Tele-Ophthalmology Society — organizer of the source Kaggle competition/dataset.
SRS	Software Requirements Specification — this document.
Fundus image	A photograph of the interior surface of the eye (retina), used to detect DR.

1.5 References

- APTOS 2019 Blindness Detection dataset — Kaggle competition page.

- TensorFlow / Keras documentation — https://www.tensorflow.org
- Streamlit documentation — https://docs.streamlit.io
- NAVTTC Assignment 7 — Capstone Mini-Project (ML/DL) brief.

2. Problem Statement

Diabetic Retinopathy is one of the leading causes of preventable blindness worldwide. Detecting it early allows timely treatment that can prevent vision loss, but grading its severity from a fundus photograph currently requires a trained ophthalmologist manually reviewing the image. This process is slow, dependent on scarce specialist availability, and does not scale well to large-scale screening programs, particularly in low-resource or rural settings where access to ophthalmologists is limited.

This project addresses that gap by building a deep learning model that automatically estimates DR severity from a fundus image, wrapped in an accessible web application. The goal is to demonstrate a fast, low-cost, automated screening aid that could support (not replace) clinical decision-making, while providing a complete, portfolio-ready example of an end-to-end applied machine learning project.

3. Objectives

- Select and justify a real-world, publicly available dataset suited to a classification problem (APTOS 2019 Blindness Detection).
- Perform thorough exploratory data analysis (EDA) covering class balance, image quality, color composition, and structural characteristics of the dataset.
- Design and implement an image preprocessing pipeline (border cropping and local contrast enhancement) to improve model input quality.
- Build, train, and evaluate a CNN-based classifier using transfer learning (EfficientNetB0), using metrics appropriate to an imbalanced, ordinal classification problem.
- Package the trained model behind a simple, user-friendly Streamlit web application that accepts an uploaded image and returns a prediction.
- Provide interpretability aids — confidence scores, per-class probability breakdowns, and visual diagnostics — so predictions are not a black box to the user.
- Publish the complete project (notebook, code, README, and a live demo link) as a public, professional portfolio artifact.

4. Functional Requirements

The table below lists the functional requirements (FR) the system satisfies.

ID	Requirement	Description
FR-1	Image Upload	The system shall allow a user to upload a fundus image in JPG or PNG format through the web interface.
FR-2	Image Preprocessing	The system shall crop black borders from the uploaded image and apply Ben Graham-style local contrast enhancement before inference.
FR-3	Severity Prediction	The system shall classify the preprocessed image into one of five DR severity grades (0-4) using the trained EfficientNetB0 model.

ID	Requirement	Description
FR-4	Confidence Display	The system shall display the model's confidence (predicted probability) for the top predicted class.
FR-5	Probability Breakdown	The system shall display the predicted probability for every one of the five severity classes as a chart.
FR-6	Prediction Insights	The system shall provide a dedicated view showing supplementary visualizations for the current prediction: a confidence donut chart, a probability radar chart, an original-vs-preprocessed image comparison, and a pixel-intensity histogram.
FR-7	Severity Reference	The system shall display a reference table describing what each of the five severity grades means.
FR-8	Model Information	The system shall display basic information about the model architecture, training dataset, and number of classes.
FR-9	Medical Disclaimer	The system shall display a clearly visible disclaimer stating the tool is educational and not a certified medical device.
FR-10	Graceful Handling of No Input	The system shall display an informative prompt instructing the user to upload an image when no image has yet been provided.

5. Non-Functional Requirements

Category	Requirement
Performance	The system shall return a prediction within a few seconds of image upload on standard hardware (CPU or GPU). Model quality is tracked primarily via Quadratic Weighted Kappa (QWK), the metric best suited to this ordinal, imbalanced classification problem, rather than raw accuracy alone.
Usability	The interface shall require no technical or machine-learning knowledge to operate: upload an image and read the result.
Reliability	The system shall handle unsupported file types or corrupted uploads without crashing, presenting a clear message instead.
Portability	The application shall run on any platform with Python 3.10+ and a modern web browser, and shall be deployable to Streamlit Community Cloud without code changes.
Maintainability	The codebase shall separate preprocessing, model loading, and UI logic into distinct, documented functions to ease future updates or retraining.
Scalability	Each prediction request shall be stateless, allowing the application to be containerized and horizontally scaled if usage grows.
Security & Privacy	Uploaded images shall be processed in-memory for the duration of the session only and shall not be persisted to disk or transmitted to any third party.
Availability	The public demo shall be hosted on Streamlit Community Cloud for continuous access, subject to the hosting platform's own uptime guarantees.

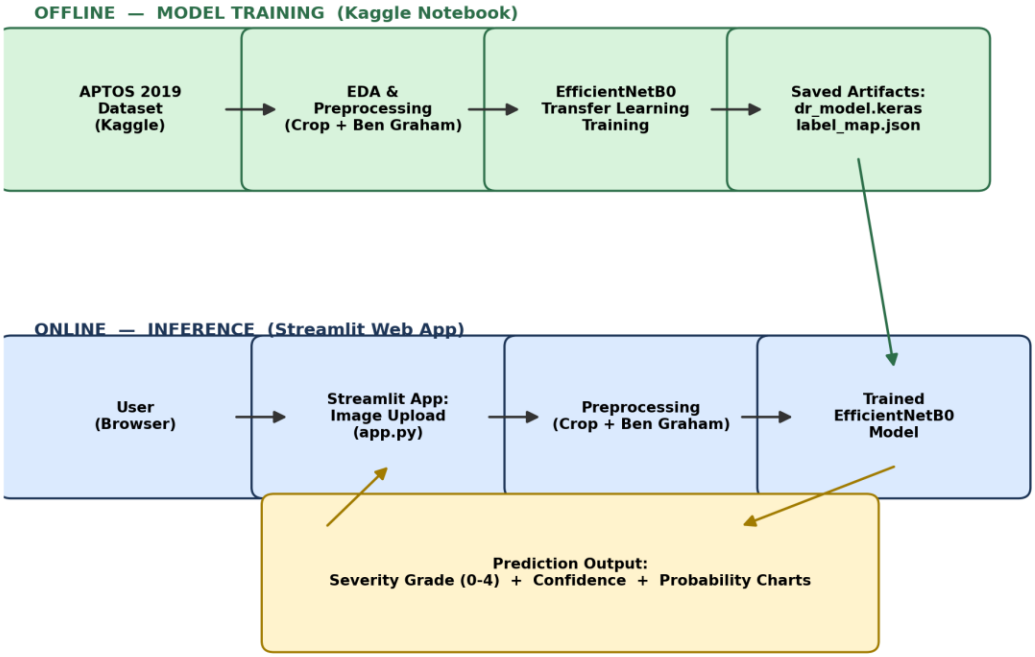
6. Tools & Technologies

Category	Tool / Technology
Programming Language	Python 3.10+
Deep Learning Framework	TensorFlow / Keras — EfficientNetB0 (transfer learning)
Image Processing	OpenCV (border cropping, contrast enhancement, resizing)
Data Handling	NumPy, Pandas
Data Visualization	Matplotlib, Seaborn
Model Evaluation	scikit-learn (classification metrics, confusion matrix, Cohen's Kappa)
Web Application Framework	Streamlit
Model Training Environment	Kaggle Notebooks (GPU-accelerated)
Application Development Environment	PyCharm
Version Control & Hosting	Git & GitHub
Deployment Platform	Streamlit Community Cloud
Dataset Source	Kaggle — APTOS 2019 Blindness Detection competition

7. System Design Overview

7.1 Architecture Diagram

The system is split into two clearly separated pipelines: an offline training pipeline (run once, on Kaggle) that produces the trained model artifacts, and an online inference pipeline (the Streamlit app) that loads those artifacts and serves predictions to end users.



7.2 Component Description

Layer	Responsibility
Data Layer	The APTOS 2019 dataset (train/test fundus images plus CSV metadata) is the sole source of training data.
Preprocessing Layer	Shared crop_image_from_gray() and ben_graham_preprocess() functions remove black borders and enhance local contrast. The same functions are used identically in both the training notebook and the deployed app, so training and inference see consistent input.
Model Layer	An EfficientNetB0 backbone (ImageNet-pretrained) with a custom classification head, trained via transfer learning on the preprocessed APTOS images and exported as dr_model.keras, alongside a label_map.json mapping class indices to human-readable severity names.
Application Layer	A Streamlit application (app.py) exposing two views: a Scan tab for uploading an image and viewing the prediction, and a Prediction Insights tab with supplementary visual diagnostics for that same prediction.
Deployment Layer	The application code and model artifacts are version-controlled in a public GitHub repository and deployed to Streamlit Community Cloud for public access.

7.3 Data Flow

- 1. The user uploads a fundus image through the Streamlit interface.
- 2. The image is preprocessed (border-cropped, resized, and contrast-enhanced) using the same pipeline used during training.
- 3. The preprocessed image is normalized and passed as a batch of one to the trained EfficientNetB0 model.

- 4. The model returns a probability distribution across the five severity classes.
- 5. The application determines the top predicted class and its confidence, and renders the result, probability chart, and (on the Insights tab) supplementary visualizations.
- 6. The user views the severity grade, confidence, and supporting charts, alongside a medical disclaimer.

8. Assumptions and Constraints

- The system assumes uploaded images are genuine retinal fundus photographs; behavior on unrelated images is undefined.
- Model performance is bounded by the size (~3,662 images) and class imbalance of the APTOS 2019 training set, particularly for the rarer Severe and Proliferative DR grades.
- The system is a proof-of-concept / educational tool and is explicitly not intended, validated, or approved for real clinical diagnostic use.
- Inference performance depends on the host environment; Streamlit Community Cloud's free tier provides CPU-only inference, which is slower than the GPU-accelerated training environment.